

Progetto 2

Introduzione

Il progetto prevede l'implementazione di una nuova versione dell'algoritmo **select**, il **sampleMedianSelect**, in cui la scelta del pivot rispetto cui effettuare la partizione ricade sul mediano di un sottoinsieme V di m elementi casuali, e di modificare l'algoritmo di ordinamento quickSort affinché la scelta del perno avvenga tramite un algoritmo di selezione.

Scelte implementative

La base usata per l'implementazione del **sampleMedianSelect** è stata l'algoritmo **quickSelectDeterministico**, che prevedeva in origine la partizione della lista di input in sottoinsiemi con al massimo 5 elementi e la scelta del mediano della lista costituita dai mediani delle quintuple come perno. Il nuovo algoritmo prevede invece la scelta di un unico **sottoinsieme V** di **m** elementi presi casualmente tra quelli di input, il cui mediano, selezionato tramite una chiamata ricorsiva dello stesso su tale sottolista, è utilizzato come pivot, richiamando così ricorsivamente la funzione su una porzione di lista opportuna. Per la creazione del sottoinsieme è stato utilizzato il metodo **random.sample** dalla libreria random di Python, il quale permette la selezione senza ripetizioni di un campione casuale di grandezza fissata a partire dalla lista di input.

Per il parametro **m** è risultato più efficiente la scelta di un valore, approssimato per eccesso, pari all'**1%** della lunghezza della porzione di lista considerata. Infatti basandoci sull'osservazione di Floyd e Rivest, secondo i quali l'algoritmo è tanto più efficiente quanto il perno è più vicino all'elemento selezionato, la scelta di tale valore per il parametro **m** è un buon compromesso fra efficienza dell'algoritmo, il qual deve essere richiamato ricorsivamente su un sottoinsieme di lunghezza minore, e buone probabilità di scegliere un pivot conveniente, come verificato sperimentalmente.

La ricorsione si ferma quando la porzione di lista considerata è sufficientemente piccola, con una chiamata al **trivialSelect**.

Per quanto riguarda la nuova implementazione del **quickSort** è stata modificata la modalità di scelta del pivot rispetto a quello classico, dato che avviene attraverso un algoritmo di selezione.

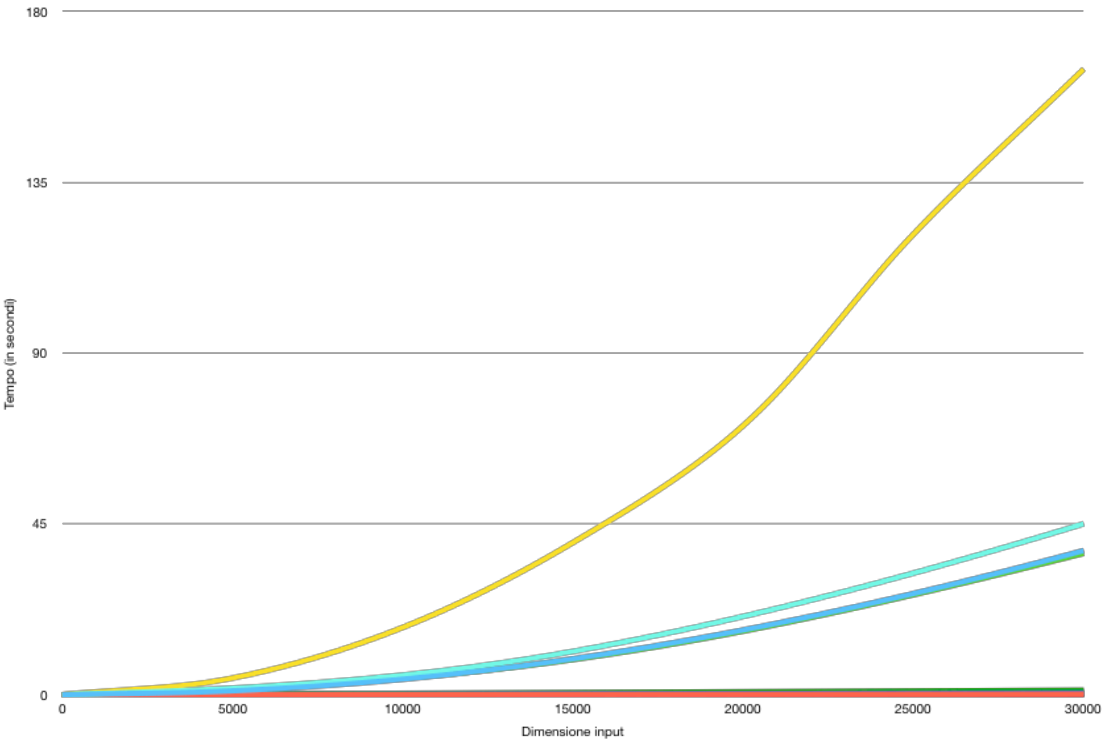
In particolare selezioniamo l'elemento in posizione k , con k pari alla parte superiore di $\frac{(right - left + 1)}{2}$ (corrispondente dunque a quella centrale), dove **right** è l'indice dell'ultimo elemento della porzione di lista considerata e **left** ne è il primo. Tale scelta del parametro k è giustificata dalla necessità di avere delle partizioni bilanciate e dunque tempi migliori di esecuzione del nuovo **quickSort** ($O(n \log n)$). Infatti se avessimo scelto come perno un elemento troppo vicino alla prima o all'ultima posizione avremmo ottenuto tempi di esecuzione quadratici. Inoltre tale algoritmo, nel caso in cui il pivot venga selezionato attraverso il **sampleMedianSelect** o il **quickSelect** deterministico, non effettua l'ordinamento in loco, poiché, nella chiamata a una di queste funzioni per la selezione del pivot, si utilizza una lista ausiliaria per l'esecuzione degli stessi.

Risultati sperimentali:

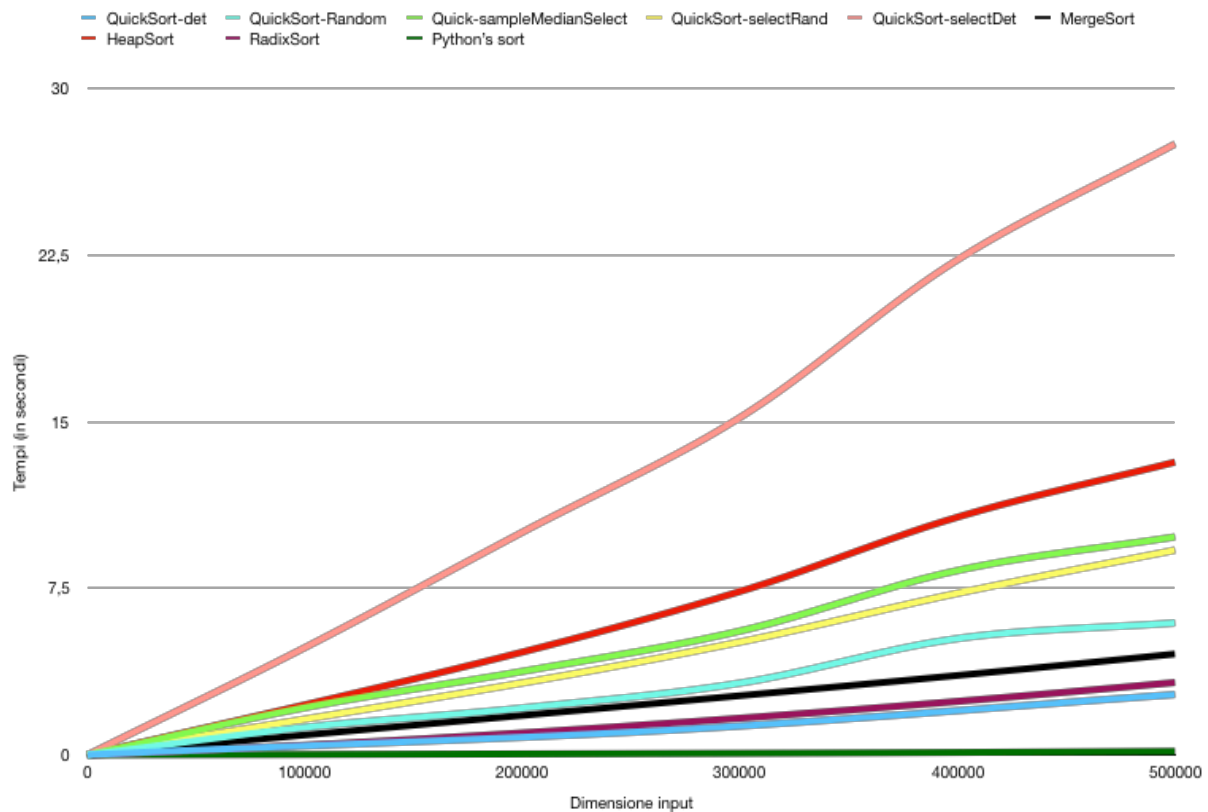
Test con input di elementi random

	SelectionSort	InsertionSortUp	InsertionSortDown	BubbleSort	QuickSort-det	QuickSort-Random	Quick-sampleMedian	quickSort-selectRandom	quickSort-selectDet	mergeSort	heapSort	radixSort	Python's sort
0	0	0	0	0	0	0	0	0	0	0	0	0	0
5000	1,01273 202896 11816	1,26599 812507 6294	1,01228 308677 67334	4,38793 802261 352	0,01464 9868011 47461	0,021157 0262908 93555	0,07090 7115936 2793	0,06622 2906112 6709	0,16273 1170654 29688	0,030655 8609008 78906	0,08003 1394958 4961	0,017968 8930511 4746	0,000619 1730499 267578
10000	4,07409 214973 4497	5,01868 987083 4351	4,09307 718276 9775	17,7202 019691 4673	0,03119 6832656 86035	0,045472 8603363 0371	0,14548 1824874 87793	0,13520 7176208 4961	0,33653 2831192 0166	0,063700 9143829 3457	0,06370 0914382 93457	0,035754 9190521 2402	0,001288 4140014 648438
15000	9,40949 869155 8838	11,4017 021656 03638	9,25171 995162 9639	40,0863 280296 3257	0,05052 5188446 04492	0,068291 9025421 1426	0,23713 2072448 73047	0,20219 7790145 87402	0,54295 3014373 7793	0,097688 9133453 3691	0,27656 8174362 1826	0,051522 2549438 47656	0,002048 0155944 82422
20000	17,0612 571239 47144	20,9814 999103 54614	16,6280 438899 9939	70,8901 491165 1611	0,06590 4140472 41211	0,087769 2699432 373	0,30761 5995407 1045	0,27858 2096099 8535	0,73869 4906234 7412	0,136090 7554626 4648	0,36962 7237319 9463	0,068819 9996948 2422	0,002982 1395874 023438
25000	25,8712 658882 1411	31,4306 900501 25122	25,9131 741523 74268	121,305 493831 63452	0,08463 6926651 00098	0,124435 9016418 457	0,41454 4105529 78516	0,35358 7865829 4678	0,98142 3854827 8809	0,172390 9378051 7578	0,47775 0062942 5049	0,085258 9607238 7695	0,003645 1816558 83789
30000	37,9724 378585 8154	45,0255 911350 25024	37,2357 752323 1506	164,802 376031 8756	0,10932 6839447 02148	0,149716 8540954 5898	0,48776 5073776 2451	0,43323 0876922 6074	1,18659 4963073 7305	0,217453 9566040 039	0,59390 2111053 4668	0,104654 7889709 4727	0,004559 9937438 96484

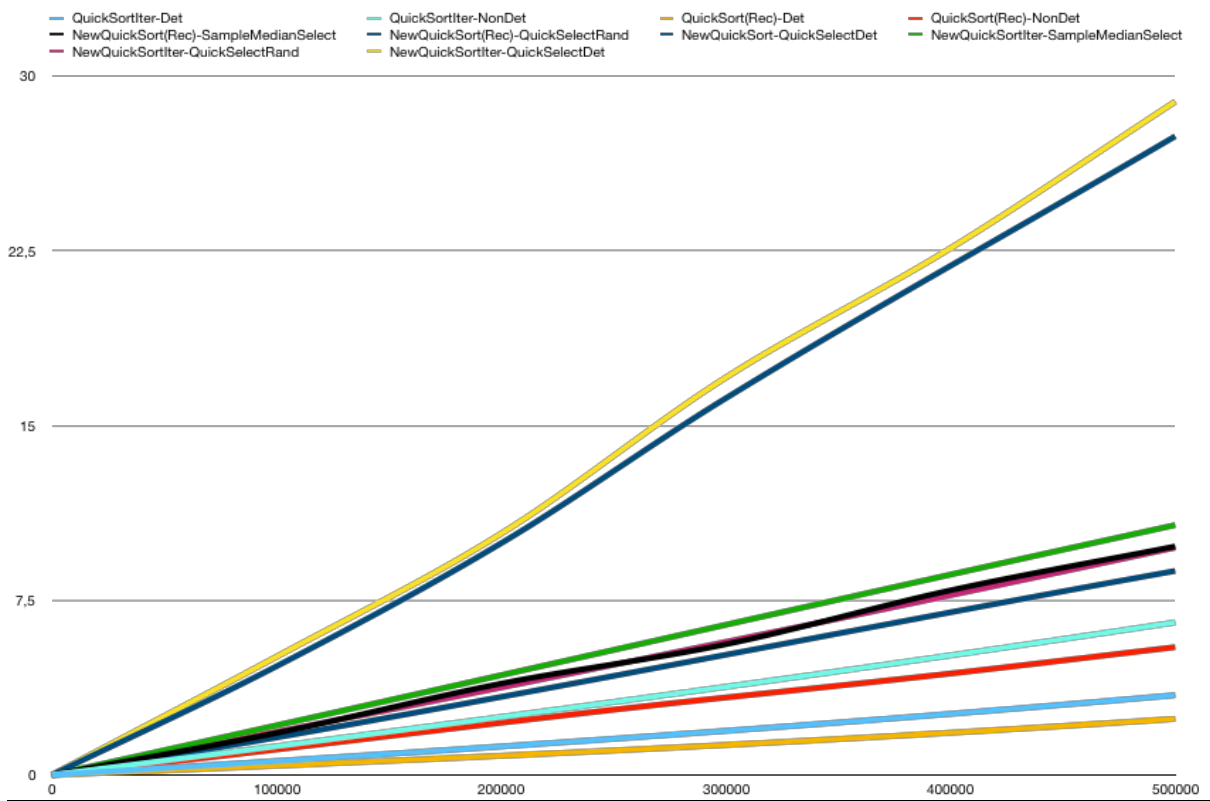
SelectionSort InsertionSortUp InsertionSortDown BubbleSort QuickSort-det QuickSort-Random
Quick-sampleMedian quickSort-selectRand quickSort-selectDet mergeSort heapSort radixSort
Python's sort



	QuickSort-det	QuickSort-Random	Quick-sampleMedianSelect	QuickSort-selectRand	QuickSort-selectDet	MergeSort	HeapSort	RadixSort	Python's sort
0	0	0	0	0	0	0	0	0	0
100 000	0,471346855 1635742	1,241288185 119629	2,126486063 00354	1,659686088 5620117	4,831838846 206665	0,913729906 0821533	2,271693944 9310303	0,377754926 68151855	0,0212848186 49291992
200 000	0,812329053 8787842	2,113641023 6358643	3,760553121 5667725	3,265999078 7506104	10,02151393 8903809	1,666300058 3648682	4,734665870 666504	1,116571903 2287598	0,0418460369 1101074
300 000	1,302095890 045166	3,233099937 438965	5,575408935 546875	5,079683065 414429	15,18139791 4886475	2,594918966 293335	7,354419946 670532	1,736999034 8815918	0,0710587501 5258789
400 000	2,037503004 0740967	5,236330986 022949	8,297628164 291382	7,273411035 53772	22,27585673 3322144	3,547364234 9243164	10,69916415 2145386	2,367490053 17688	0,1058931350 7080078
500 000	2,715512990 951538	5,940711736 679077	9,810347080 230713	9,214051008 224487	27,48766422 2717285	4,542561769 485474	13,17489314 0792847	3,259333848 953247	0,1646599769 5922852



	QuickSortIter-Det	QuickSortIter-NonDet	QuickSort(Rec)-Det	QuickSort(Rec)-NonDet	QuickSort(Rec)-SampleMedianSelect	QuickSort(Rec)-QuickSelectRand	QuickSort(Rec)-QuickSelectDet	QuickSortIter-SampleMedianSelect	QuickSortIter-QuickSelectRand	QuickSortIter-QuickSelectDet
0	0	0	0	0	0	0	0	0	0	0
100 000	0,5838322 639465332	1,2231252 193450928	0,3877723 217010498	1,0238680 839538574	1,801234006 8817139	1,5290360 45074463	4,6647741 79458618	2,10731387 1383667	1,8193500 04196167	5,0764679 90875244
200 000	1,2592058 181762695	2,5908288 955688477	0,8864121 437072754	2,3279349 80392456	3,9228899 478912354	3,2976338 863372803	9,9479680 06134033	4,40904688 835144	3,7495007 514953613	10,363355 159759521
300 000	1,9340901 374816895	3,7449769 973754883	1,2790310 382843018	3,3235461 711883545	5,605927944 18335	5,3248229 02679443	16,172825 813293457	6,40948915 4815674	5,7780060 76812744	17,061592 10205078
400 000	2,5830097 19848633	5,0970752 239227295	1,7724657 05871582	4,3529541 49246216	7,925710916 519165	7,0209550 857543945	21,764864 921569824	8,62047576 9042969	7,6959819 79370117	22,613500 833511353
500 000	3,4231760 501861572	6,5473301 41067505	2,4056570 529937744	5,4852690 69671631	9,812685012 817383	8,7579720 02029419	27,418272 972106934	10,7260630 13076782	9,7606248 85559082	28,905811 071395874

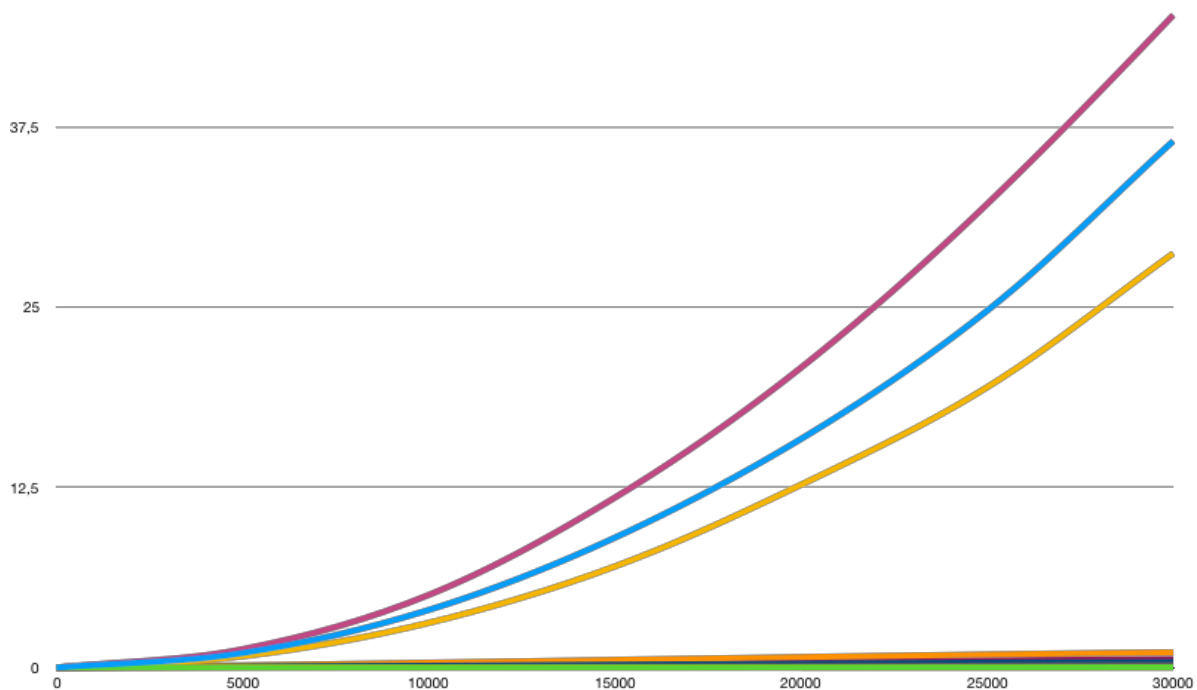


Test con input di elementi crescenti

	Selecti onSort	Insertio nSortUp	Insertio nSortDown	BubbleS ort	QuickS ort-det	QuickSo rt-Random	Quick- sample Median	quickSo rt- selectR and	quickS ort- selectD et	mergeS ort	heapSo rt	Python's sort	radixSor t
0	0	0	0	0	0	0	0	0	0	0	0	0	0
5000	1,01002 907752 99072	0,00394 6065902 709961	0,78294 301033 02002	0,000584 8407745 361328	1,27262 306213 3789	0,01752 4003982 543945	0,05657 7205657 958984	0,05214 5957946 777344	0,13050 389289 855957	0,02634 9067687 98828	0,08406 615257 263184	3,361701 96533203 1*10 [^] (-5)	0,01751 4944076 538086
10000	3,99722 695350 64697	0,00759 6015930 175781	3,10237 073898 31543	0,001116 7526245 117188	5,03934 311866 76025	0,03620 1000213 62305	0,11604 9766540 52734	0,11051 8693923 9502	0,28654 623031 61621	0,05462 4080657 958984	0,18146 181106 567383	6,222724 91455078 1*10 [^] (-5)	0,03466 6061401 36719
15000	8,98812 007904 0527	0,01120 7342147 827148	7,01906 991004 94385	0,001680 1357269 28711	11,7951 169013 97705	0,05684 9241256 71387	0,18536 4007949 8291	0,17167 3297882 08008	0,47895 503044 12842	0,08075 4280090 33203	0,28078 579902 648926	9,298324 58496093 8*10 [^] (-5)	0,05091 7148590 08789
20000	16,0081 539154 05273	0,01512 8135681 152344	12,6863 098144 53125	0,002236 8431091 308594	20,7070 908546 44775	0,07771 8019485 47363	0,24692 5830841 06445	0,22726 2973785 4004	0,63966 202735 90088	0,11309 3137741 08887	0,39357 471466 064453	0,000108 24203491 210938	0,06707 8828811 64551
25000	24,7669 849395 75195	0,01903 5100936 88965	19,4602 279663 08594	0,002791 1663055 41992	31,7037 019729 61426	0,13880 0859451 29395	0,34031 4149856 5674	0,28198 9097595 21484	0,92486 500740 05127	0,14172 6016998 29102	0,49702 405929 56543	0,000161 17095947 265625	0,08365 1065826 41602
30000	36,5454 111099 24316	0,02281 8088531 49414	28,7460 312843 32275	0,003482 1033477 783203	45,2861 428260 8032	0,17422 4853515 625	0,38591 8855667 11426	0,33437 8957748 4131	1,05371 809005 7373	0,16881 2036514 28223	0,58719 491958 61816	0,000154 25682067 871094	0,10113 4061813 35449

— SelectionSort
 — InsertionSortUp
 — InsertionSortDown
 — BubbleSort
 — QuickSort-det
 — QuickSort-Random
 — Quick-sampleMedian
 — quickSort-selectRand
 — quickSort-selectDet
 — mergeSort
 — heapSort
 — Python's sort
 — radixSort

50

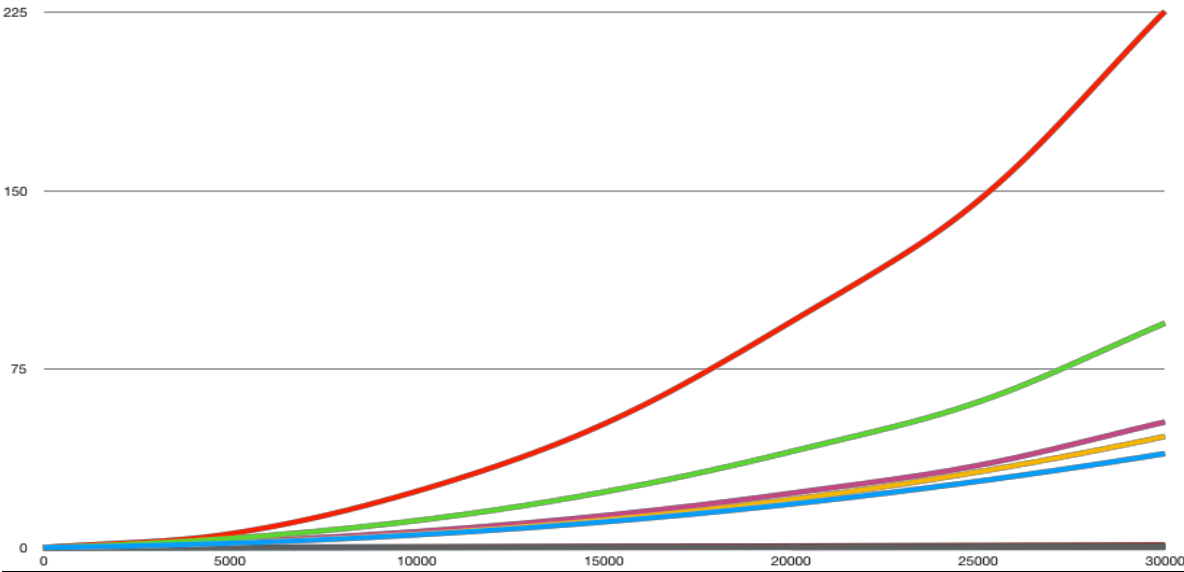


Test con input di elementi decrescenti

	Selecti onSort	Insertio nSortUp	Insertio nSortDown	Bubble Sort	QuickS ort-det	QuickS ort-Rando m	Quick- sample Median	quickSo rt- selectRa nd	quickSo rt- selectD et	mergeS ort	heapSo rt	Python's sort	radixSor t
0	0	0	0	0	0	0	0	0	0	0	0	0	0
5000	1,06032 514572 14355	2,44139 909744 2627	1,18443 894386 2915	5,83048 605918 8843	1,47102 499008 1787	0,02698 2069015 50293	0,05674 0045547 48535	0,052416 0861968 99414	0,14135 5037689 20898	0,025829 0767669 67773	0,07710 5283737 18262	3,0994415 28320312 5*10 [^] (-5)	0,017619 8482513 42773
10000	4,24803 590774 5361	10,1671 769618 98804	5,13489 294052 124	23,7614 071369 17114	5,82977 795600 8911	0,05452 0130157 4707	0,11919 8083877 56348	0,109691 8582916 2598	0,29129 2905807 4951	0,056474 2088317 8711	0,17387 7000808 71582	5,9127807 6171875* 10 [^] (-5)	0,036745 0714111 3281
15000	9,54213 309288 0249	22,4661 300182 34253	11,2848 191261 2915	52,0246 558189 3921	12,8895 919322 96753	0,08432 1022033 6914	0,19213 9863967 8955	0,157861 2327575 6836	0,49197 1254348 7549	0,083398 8189697 2656	0,25661 7069244 38477	8,2969665 52734375 *10 [^] (-5)	0,051465 9881591 7969
20000	17,4413 020610 80933	40,3685 078620 91064	20,0011 079311 37085	94,9343 190193 1763	22,8448 719978 33252	0,11231 6131591 79688	0,24638 9150619 50684	0,234299 8981475 83	0,65803 8139343 2617	0,113131 9999694 8242	0,36272 2635269 16504	0,0001311 30218505 85938	0,072047 7104187 0117
25000	26,8799 090385 437	61,2055 299282 074	30,0978 739261 6272	145,806 809663 77258	34,5536 930561 0657	0,14439 9881362 91504	0,34239 0775680 542	0,295331 2397003 174	0,92987 1082305 9082	0,138377 9048919 6777	0,44978 6901473 999	0,0001442 43240356 4453	0,083891 8685913 086
30000	39,5103 271007 53784	94,3748 519420 6238	46,7293 820381 16455	225,516 680955 88684	52,8356 311321 25854	0,19233 0598831 17676	0,42135 5962753 2959	0,386384 9639892 578	1,15004 3725967 4072	0,179283 1420898 4375	0,59591 6032791 1377	0,0001928 80630493 16406	0,108458 2805633 5449

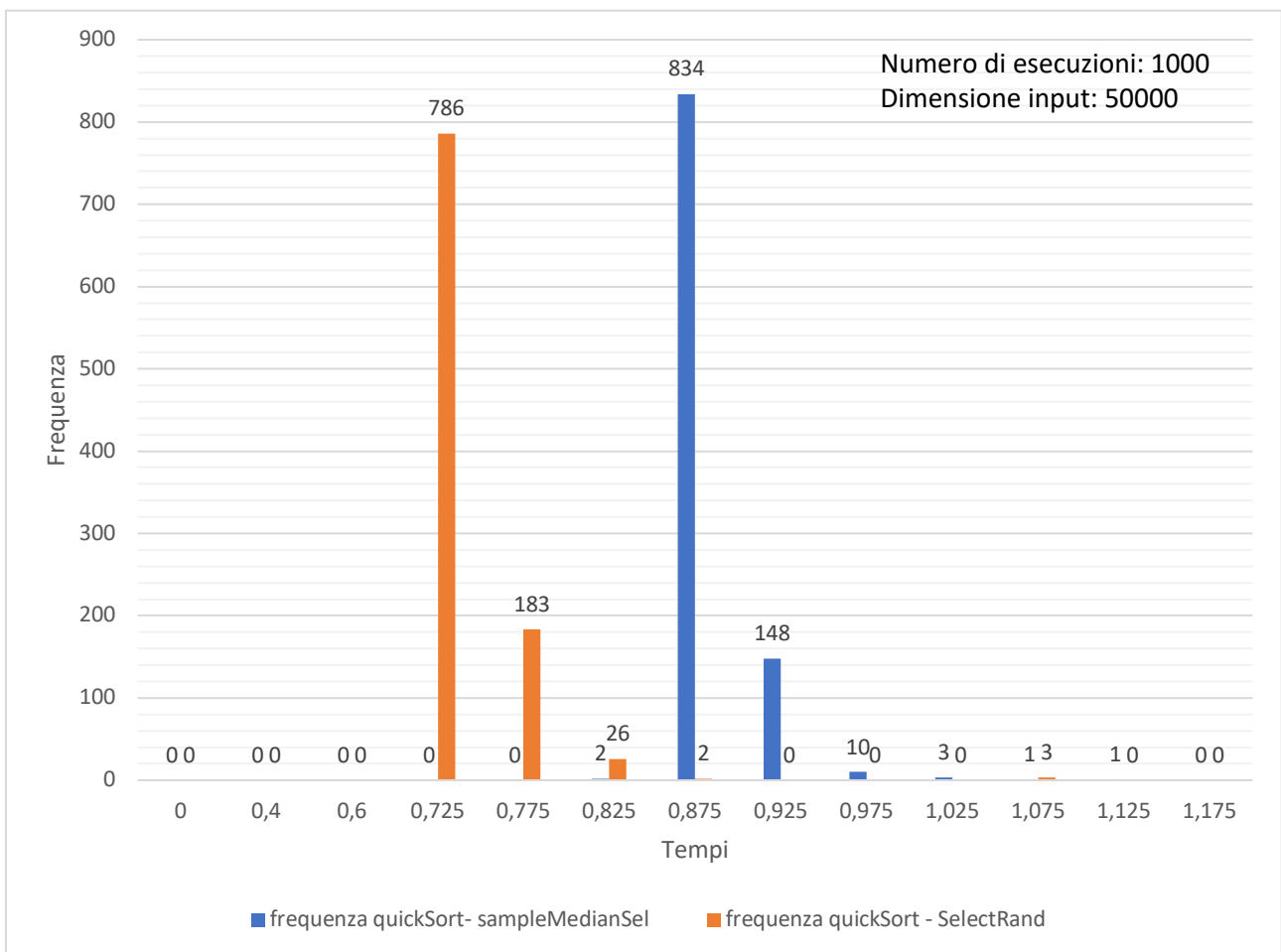
SelectionSort InsertionSortUp InsertionSortDown BubbleSort QuickSort-det QuickSort-Random Quick-sampleMedian
quickSort-selectRand quickSort-selectDet mergeSort heapSort Python's sort radixSort

300



Tempi di esecuzione nuovi QuickSort (no select deterministico)

tempi	frequenza quickSort- sampleMedianSel	frequenza quickSort - SelectRand
0	0	0
0,4	0	0
0,6	0	0
0,725	0	786
0,775	0	183
0,825	2	26
0,875	834	2
0,925	148	0
0,975	10	0
1,025	3	0
1,075	1	3
1,125	1	0
1,175	0	0
	tempo medio	tempo medio
	0,884209	0,738100



Commenti dei risultati ottenuti:

Come si può evincere dai test effettuati, le nuove varianti del **quickSort** che scelgono il pivot, rispetto cui ripartire l'insieme, attraverso un algoritmo di selezione sono da 2 fino a 9 volte più lenti del **quickSort classico**. In particolare, quella che si avvale del **select deterministico** è risultata la più inefficiente fra tutte, con tempi di esecuzione di 3 volte superiori rispetto alle altre due varianti, fino a 9 volte superiori rispetto a quello classico deterministico e di 4 volte rispetto a quello classico randomizzato. La minore efficienza di tali algoritmi è da giustificarsi nella necessità di invocare un'ulteriore funzione per scegliere il pivot, con conseguente ripartizione dell'insieme due volte: la prima infatti avviene rispetto al perno preso in considerazione durante l'esecuzione di uno degli algoritmi di selezione e la seconda rispetto al pivot selezionato tramite questi ultimi. Dunque se da una parte si ha un guadagno scegliendo come pivot l'elemento in posizione centrale così da avere due sottoinsiemi bilanciati su cui operare attraverso il **quickSort**, dall'altra la concomitante esecuzione dell'algoritmo di selezione, non presente nel **quickSort classico**, porta a tempi di esecuzione leggermente maggiori. Ciò si nota in particolar modo con l'implementazione del **quickSelect deterministico**, in cui la scelta del pivot ricade sul mediano di un campione costituito dai mediani delle quintuple dell'insieme di input, con tempi di esecuzione di quest'ultimo, come si può verificare, di circa 9 volte superiori rispetto al **select randomizzato** o al **sampleMedianSelect**.

In definitiva, nel caso di un input di elementi random, secondo i test effettuati, tra tutte le varianti del **quickSort** è risultato essere più efficiente quello **classico deterministico**, il quale, però, è molto inefficiente nel caso di input di elementi crescenti e decrescenti, dal momento che, per come è stato implementato, sceglie rispettivamente l'elemento più piccolo o quello più grande come perno rispetto a cui effettuare la partizione dell'insieme, con tempi di esecuzione dunque, come già dimostrato, quadratici nel caso peggiore.

Il metodo **sort di Python** è risultato di gran lunga il più efficiente in tutte le situazioni.

Con input di elementi casuali o decrescenti **bubbleSort** è il meno efficiente, dato che deve confrontare elementi adiacenti fra loro ad ogni scansione dell'insieme di ingresso, con tempo di esecuzione quadratico nel caso peggiore, mentre se gli elementi sono ordinati in modo crescente lo è **quickSort deterministico** (per i motivi soprammenzionati).

